AUTOMOTIVE DATA PIPELINE: INGESTION, TRANSFORMATION, AND ANALYSIS USING DATABRICKS

Overview

This document overviews a data pipeline that processes automotive datasets from NHTSA, AFDC, and EPA using Databricks and the Medallion architecture. Gold-layer datasets and dashboards provide actionable insights for vehicle safety, infrastructure planning, and sustainability, using a reliable hybrid of manual and automated processing.

Daniel Miat

Table of Contents:

1. Executive Summary

Objective

- Build a data pipeline (preferably in Python or SQL) to acquire, process, transform, and load data from public automotive industry sources.

Tasks

- **Data Acquisition:**
  Write scripts to download datasets from specified public sources.
- **Data Processing:**
  Clean and integrate datasets, handling missing values, duplicates, and outliers.
- **Data Transformation:**
  Convert data into an analysis-ready format and justify your transformation choices.
- **Data Loading:**
  Simulate loading data into a storage system (e.g., Azure SQL Database or Databricks Delta Lake) using relevant commands.
- **Automation:**
  Describe how you would automate the pipeline, including scheduling and your reasoning.

Data Sources

- U.S. Department of Transportation – National Highway Traffic Safety Administration (Vehicle Complaints)
- U.S. Department of Energy (Alternative Fuel Stations)
- U.S. Environmental Protection Agency (Vehicle Fuel Economy Information)

Documentation Requirements

- Prepare a PowerPoint presentation that explains:
  - Script functions and execution
  - Data processing and transformation decisions
  - Data loading and automation strategy
  - Challenges faced and solutions

Delivery

- Upload all scripts, processed data, and the presentation to a public GitHub repository and share the link.

2. Project Overview

Description of the Automotive Data Sources:

- **NHTSA Vehicle Complaints:** Public dataset from the U.S. Department of Transportation, containing consumer-reported vehicle safety complaints. NHTSA Datasets and APIs | NHTSA
- **DOE Alternative Fuel Stations:** Dataset from the U.S. Department of Energy, listing locations and details of alternative fuel stations across the U.S. Alternative Fuels Data Center: Data Downloads
- **EPA Vehicle Fuel Economy:** Dataset from the U.S. Environmental Protection Agency, providing fuel economy information for vehicles sold in the U.S. Fuel Economy Web Services

Motivation for Analyzing these Datasets:

- To identify safety trends and recurring issues in the automotive industry.
- To support data-driven decisions for product quality, recalls, and regulatory compliance.
- To enable insights into alternative fuel infrastructure and vehicle efficiency, supporting sustainability and innovation.
- To provide a foundation for advanced analytics, reporting, and automation in automotive data engineering

Content description – how to find files, scripts:

- Script source GIT: miatdaniel/databricks_project: Databricks personal projects

Content Description – How to Find Files and Scripts in the Repository

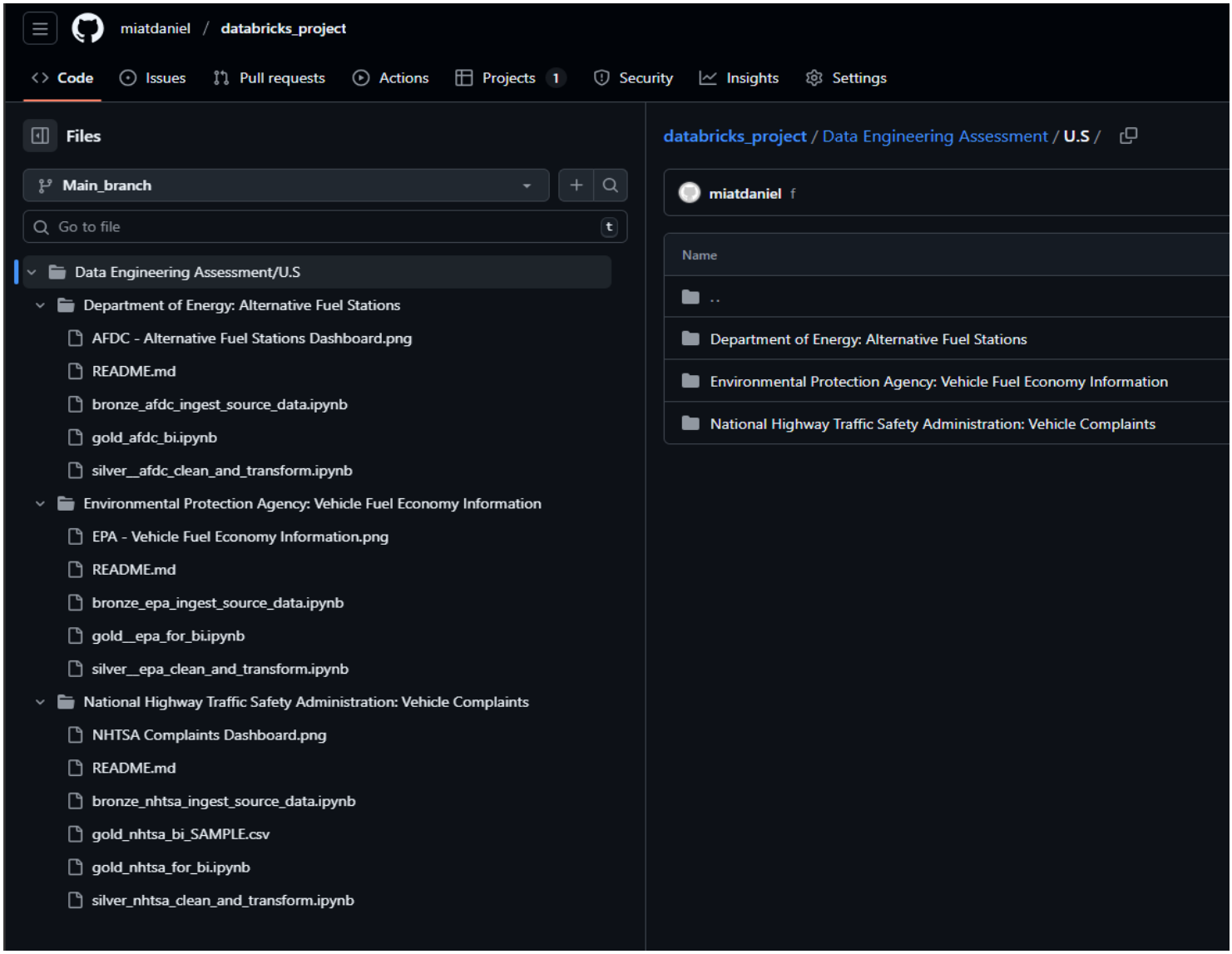
Each data source (NHTSA, AFDC, EPA) has its own structured folder inside the repository.

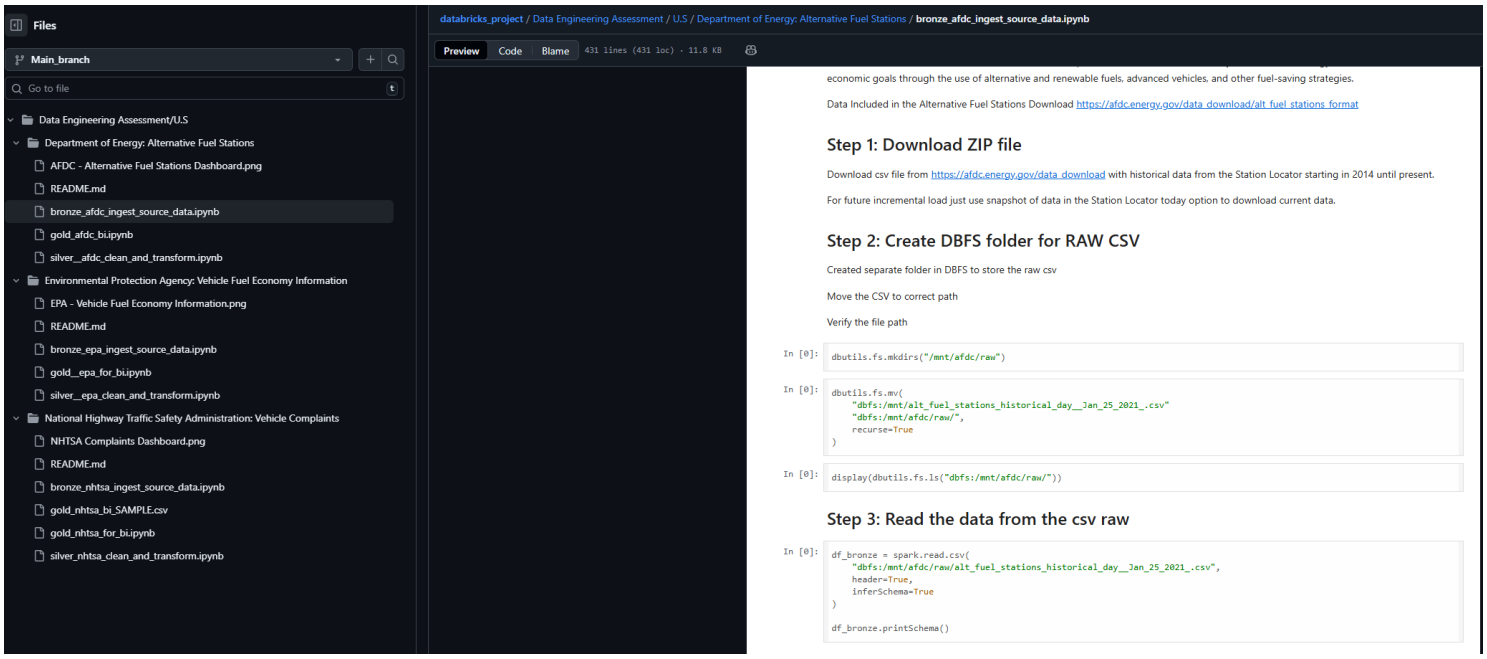
Inside each folder, you will find:

- Databricks notebooks containing the full code, with Markdown comments explaining each step.
- Exported CSV files with sample data from the gold layer.
- Documentation files (README or Markdown) describing the workflow and business logic.
- Dashboards (as images) reflecting business cases.

To locate a specific script or dataset:

- Navigate to the folder named after the data source.
- Open the notebook for code and step-by-step explanations.
- Check the CSV files for processed data samples.
- Review the Markdown/README for context and usage instructions.
- The repository structure is designed for easy navigation, so each component is clearly labeled by data source and processing stage (bronze, silver, gold).


miatdaniel / databricks_project
Code
Issues
Pull requests
Actions
Projects 1
Security
Insights
Settings
Files
Main_branch
Go to file
Data Engineering Assessment/U.S
Department of Energy: Alternative Fuel Stations
AFDC - Alternative Fuel Stations Dashboard.png
README.md
bronze_afdc_ingest_source_data.ipynb
gold_afdc_bi.ipynb
silver__afdc_clean_and_transform.ipynb
Environmental Protection Agency: Vehicle Fuel Economy Information
EPA - Vehicle Fuel Economy Information.png
README.md
bronze_epa_ingest_source_data.ipynb
gold__epa_for_bi.ipynb
silver__epa_clean_and_transform.ipynb
National Highway Traffic Safety Administration: Vehicle Complaints
NHTSA Complaints Dashboard.png
README.md
bronze_nhtsa_ingest_source_data.ipynb
gold_nhtsa_bi_SAMPLE.csv
gold_nhtsa_for_bi.ipynb
silver_nhtsa_clean_and_transform.ipynb
databricks_project / Data Engineering Assessment / U.S /
miatdaniel f
Name
..
Department of Energy: Alternative Fuel Stations
Environmental Protection Agency: Vehicle Fuel Economy Information
National Highway Traffic Safety Administration: Vehicle Complaints


Files
Main_branch
Go to file
Data Engineering Assessment/U.S
Department of Energy: Alternative Fuel Stations
AFDC - Alternative Fuel Stations Dashboard.png
README.md
bronze_afdc_ingest_source_data.ipynb
gold_afdc_bi.ipynb
silver__afdc_clean_and_transform.ipynb
Environmental Protection Agency: Vehicle Fuel Economy Information
EPA - Vehicle Fuel Economy Information.png
README.md
bronze_epa_ingest_source_data.ipynb
gold__epa_for_bi.ipynb
silver__epa_clean_and_transform.ipynb
National Highway Traffic Safety Administration: Vehicle Complaints
NHTSA Complaints Dashboard.png
README.md
bronze_nhtsa_ingest_source_data.ipynb
gold_nhtsa_bi_SAMPLE.csv
gold_nhtsa_for_bi.ipynb
silver_nhtsa_clean_and_transform.ipynb
databricks_project / Data Engineering Assessment / U.S / Department of Energy: Alternative Fuel Stations / bronze_afdc_ingest_source_data.ipynb
Preview
Code
Blame
431 lines (431 loc) · 11.8 KB
economic goals through the use of alternative and renewable fuels, advanced vehicles, and other fuel-saving strategies.
Data Included in the Alternative Fuel Stations Download https://afdc.energy.gov/data_download/alt_fuel_stations_format
Step 1: Download ZIP file
Download csv file from https://afdc.energy.gov/data_download with historical data from the Station Locator starting in 2014 until present.
For future incremental load just use snapshot of data in the Station Locator today option to download current data.
Step 2: Create DBFS folder for RAW CSV
Created separate folder in DBFS to store the raw csv
Move the CSV to correct path
Verify the file path
In [0]:
dbutils.fs.mkdirs("/mnt/afdc/raw")
In [0]:
dbutils.fs.mv(
"dbfs:/mnt/alt_fuel_stations_historical_day__Jan_25_2021_.csv"
"dbfs:/mnt/afdc/raw/",
recurse=True
)
In [0]:
display(dbutils.fs.ls("dbfs:/mnt/afdc/raw/"))
Step 3: Read the data from the csv raw
In [0]:
df_bronze = spark.read.csv(
"dbfs:/mnt/afdc/raw/alt_fuel_stations_historical_day__Jan_25_2021_.csv",
header=True,
inferSchema=True
)
df_bronze.printSchema()

Acronyms used in this project explained:

NHTSA: National Highway Traffic Safety Administration
(U.S. government agency responsible for vehicle safety and consumer protection, provides data on vehicle complaints and recalls)

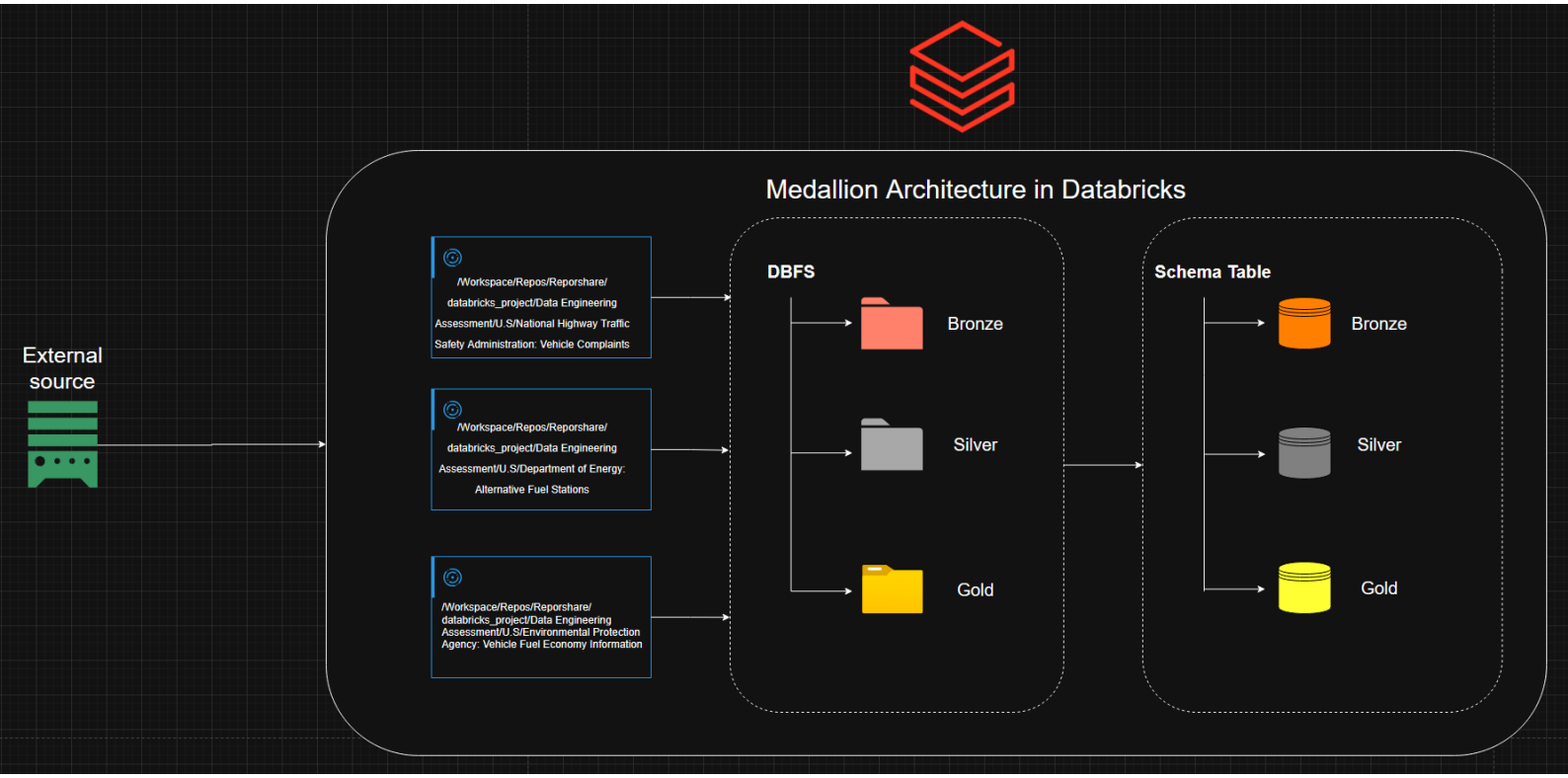
AFDC: Alternative Fuels Data Center
(A resource from the U.S. Department of Energy offering data on alternative fuel stations and clean transportation)

EPA: Environmental Protection Agency
(U.S. government agency focused on environmental protection, provides vehicle fuel economy and emissions data)

Tool Choice: Databricks: https://www.databricks.com/
Why?

- It supports Python and SQL natively.
- Ideal for data engineering workflows: ingestion, cleaning, transformation, and loading.
- Scales well for large datasets and integrates with cloud storage.
- Built-in Delta Lake for structured storage simulation.
- Easy to automate with Databricks Jobs and scheduling.

Design Choice: Databricks Medallion Architecture.



Why?

- Structured Data Flow: Organizes data into Bronze (raw), Silver (cleansed), Gold (aggregated) layers for easy management.
- Scalability & Performance: Each layer optimizes storage and compute for fast queries and efficient resource use.
- Data Quality: Silver layer cleans and transforms data for higher quality analytics in Gold.
- Flexibility: Data can be reprocessed or updated at any stage without affecting the whole pipeline.
- Auditability: Tracks changes layer by layer for compliance and debugging.
- Best Practice: Aligns with modern data engineering standards, especially in cloud platforms like Databricks.

Legend:

Bronze (orange icon): Raw data layer – contains unprocessed data directly ingested from the source.
Silver (gray icon): Cleansed data layer – data has been cleaned, transformed, and is ready for analysis.
Gold (yellow icon): Aggregated data layer – data is fully processed, enriched, and optimized for reporting or business analytics.

3. Objective & Tasks

Build a robust data pipeline to acquire, process, transform, and load automotive datasets using Databricks.

Tasks:

- Acquire data from public sources (NHTSA, AFDC, EPA).
- Clean and integrate datasets.
- Transform data for analysis.
- Simulate loading into modern storage (Delta Lake). -

  Observation: with partial access to a Databricks workspace (Catalog not enabled), I was able to create and store the results in actual tables.
- Describe how you would automate the pipeline, including scheduling and your reasoning.

4. Data Acquisition / 5. Data Processing / 6. Data Transformation / 7. Data Loading

GIT Repo for NHTSA

Data Sources:

- **U.S. Department of Transportation – National Highway Traffic Safety Administration (NHTSA):**
  Vehicle Complaints database (https://www.nhtsa.gov/nhtsa-datasets-and-apis)
  Used to identify safety issues, monitor recalls, and analyze defect trends.

Bronze Layer

Overview: Build a raw ingestion pipeline for NHTSA vehicle complaint data using publicly available datasets.

1. **Download ZIP file**
   - Action: Use requests to stream and save the ZIP to /dbfs/tmp/nhtsa_complaints_raw/FLAT_CMPL.zip.
     Maintains a reproducible snapshot of the raw external dataset.
2. **Extract ZIP contents**
   - Action: Use zipfile to extract all files into /dbfs/tmp/nhtsa_complaints_raw/flat_cmpl.
     Prepares the raw text file for Spark ingestion without altering its structure.
3. **Read raw text file & inspect schema**
   - Action: Load FLAT_CMPL.txt with inferSchema and no header.
     Provides an initial view of how Spark interprets field types before applying a defined schema.
4. **Define the official NHTSA schema**
   - Action: Create a StructType reflecting the CMPL.txt standard.
     Ensures the ingested data aligns with the authoritative specification provided by NHTSA.
5. **Read file using defined schema + add timestamp**
   - Action: Load with .schema(schema) and append Data_update_timestamp = current_timestamp().
     Captures ingestion time for lineage, incremental logic, and downstream auditing.
6. **Write DataFrame to Bronze Delta table**

- Action: Save to dbfs:/mnt/datalake/bronze/nhtsa_complaints with Delta + mergeSchema.
  Stores raw-but-structured data with ACID guarantees and schema evolution.

7. **Register Bronze table in metastore**
   - Action: Create nhtsa_complaints.bronze referencing the Delta path.
     Enables SQL access and supports downstream ETL in a consistent environment.
8. **Preview Bronze table**
   - Action: Display the SQL table.
     Confirms successful ingestion with the expected schema and rows.

Silver Layer

Overview: Clean, standardize, and validate Bronze data while retaining all fields and preserving analytical usefulness.

1. **Enable Delta auto-merge**
   - Action: Turn on schema evolution.
     Allows safe incorporation of schema changes during Silver processing.
2. **Read Bronze data**
   - Action: Load nhtsa_complaints.bronze into a DataFrame.
     Establishes a stable starting point for all data quality operations.
3. **Trim whitespace & normalize strings**
   - Action: Apply trim() to all string columns.
     Reduces formatting inconsistencies that affect joins, comparisons, and grouping.
4. **Handle NULL values with safe defaults**
   - Action: Replace nulls by type:
     - Strings → "UNKNOWN"
     - Integers → -1
     - Floats/Decimals → -1.0
     - Boolean → False
       Ensures all columns remain intact without dropping fields, while creating consistent missing-data indicators.
5. **Check for duplicates using CMPLID**
   - Action: Group by CMPLID and detect counts > 1.
     Validates uniqueness of NHTSA's complaint identifier.

6. **Repartition for balanced output**
    - Action: Repartition to at least 32 or cores × 8.
      Produces efficient file sizes, improving read/write performance and later optimizations.
7. **Write Silver Delta table with schema evolution**
    - Action: Save to /mnt/delta/silver_nhtsa with mergeSchema and register as nhtsa_complaints.silver.
      Creates a standardized, quality-checked dataset ready for analytics and modeling.
8. **Preview Silver**
    - Action: Display the table.
      Confirms formatting, cleanup, and column consistency.

Gold Layer

Overview: Produce a clean, analysis-ready dataset with fully typed fields and a consistent reporting-friendly schema.

1. **Apply final schema and cast fields**
    - Action: Convert categorical fields to string, numeric fields to integers, and dates using to_date("yyyyMMdd"). Convert timestamp column properly.
      Creates a final structure aligned with analytics tools, BI models, and time-based queries.
2. **Write Gold Delta table with schema enforcement**
    - Action: Save to /mnt/delta/gold_nhtsa with mergeSchema and register as nhtsa_complaints.gold.
      Establishes the consumer-ready dataset designed for reporting, dashboards, and trend analysis.
3. **Preview Gold table**
    - Action: Display the table.
      Confirms correct typing and readiness for downstream insights.

GIT Repo for AFDC

Data Sources:

- **U.S. Department of Energy: Alternative Fuel Stations**
  AFDC provides data and tools to help organizations use alternative fuels and advanced vehicles. The Alternative Fuel Stations dataset includes U.S. station locations, fuel types, and operational status.

  https://afdc.energy.gov/data_download/alt_fuel_stations_format

Bronze Layer

Overview: Ingest raw AFDC station CSVs into an immutable Bronze Delta table stored on DBFS.

1. **Download ZIP / CSV**
   - Action: Obtain the historical Station Locator CSV; future loads use the current snapshot.
     Preserves the authoritative source and supports both full refreshes and incremental updates.
2. **Create DBFS RAW folder & move file**
   - Action: Create /mnt/afdc/raw and move the CSV there.
     Keeps raw files organized and ready for automated ingestion.
3. **Read CSV into Spark DataFrame**
   - Action: Use spark.read.csv(..., header=True, inferSchema=True).
     Loads the dataset into Spark for efficient processing.
4. **Add Data_update_timestamp**
   - Action: Add a timestamp column with current_timestamp().
     Captures ingestion time and supports audit, lineage and incremental logic.
5. **Normalize column names**
   - Action: Replace invalid characters with underscores.
     Ensures compatibility with SQL, Delta and Spark operations.
6. **Write Bronze Delta table**
   - Action: Save to dbfs:/mnt/afdc/bronze as Delta with schema merge.
     Provides ACID guarantees, schema evolution and time-travel functionality.
7. **Register Bronze table**

- Action: Create the table in the metastore.
  Makes the raw layer accessible to SQL tools and downstream jobs.

8. **Preview Bronze**
   - Action: Display the table.
     Confirms correct ingestion and file placement.

Silver Layer

Overview: Clean and standardize Bronze data into a typed, structured Silver DataFrame.

1. **Load Bronze**
   - Action: Read Bronze table into a DataFrame.
     Establishes a consistent starting point for cleaning.
2. **Trim string columns**
   - Action: Apply trim() to all string fields.
     Removes whitespace that can break joins and type conversions.
3. **Fix trailing and double underscores**
   - Action: Remove trailing _ and replace double underscores with single ones.
     Produces clean, coherent column names for downstream work.
4. **Convert empty strings to nulls**
   - Action: Replace empty strings in numeric/date/timestamp fields with null.
     Prevents casting failures and standardizes missing-value handling.
5. **Map boolean columns**
   - Action: Convert Y/N and true/false variants to actual boolean values.
     Ensures consistent boolean behavior for filtering and aggregations.
6. **Cast integers and decimals**
   - Action: Cast integer fields and set latitude/longitude as Decimal(9,6).
     Creates accurate numeric fields suited for analysis and geospatial work.
7. **Parse dates and timestamps**
   - Action: Convert date strings into DateType and timestamps into TimestampType.
     Enables correct temporal comparisons and sorting.
8. **Null handling by type**
   - Action: Drop rows missing critical identifiers; fill numeric with 0, booleans with False, non-critical strings with "Unknown", and apply optional sentinel dates.
     Stabilizes the dataset and ensures all required dimensions are present and usable.

9. **Detect and resolve duplicates**
    - Action: Check for repeated IDs and remove invalid entries, e.g., rows with ID = 0.
      Maintains identifier uniqueness and prevents double-counting.
10. **Column cleanup and reorder**
    - Action: Clean remaining unsafe characters and bring ID to the front; position metadata columns logically.
      Improves usability and readability for analysts and SQL users.
11. **Write Silver Delta and register**
    - Action: Save to dbfs:/mnt/afdc/silver and create the metastore table.
      Provides a validated, analytics-ready version for consumption.
12. **Preview Silver**
    - Action: Display the table.
      Verifies the structure and data quality after transformations.

Gold Layer

Overview: Produce an analysis-ready Gold table with standardized values, streamlined columns, and a clean schema.

1. **Remove unusable station records**
    - Action: Optionally filter rows where key station attributes are fully unknown.
      Focuses the gold output on meaningful, interpretable stations.
2. **Standardize all "unknown" variations**
    - Action: Replace nulls and case variants with a unified "Unknown".
      Ensures consistent categorical behavior across dashboards and queries.
3. **Drop columns containing only "Unknown"**
    - Action: Identify string columns where all values are "Unknown" and drop them.
      Reduces noise and keeps only fields that contribute information.
4. **Handle French-language columns**
    - Action: Drop all _French columns.
      Maintains a single-language table for clarity, simplicity and consistent reporting.
5. **Write Gold Delta table**
    - Action: Save to dbfs:/mnt/afdc/gold with schema overwrite and register in the metastore.
      Finalizes a clean, business-focused dataset ready for analytics, BI and ML.

GIT Repo for EPA

Data Sources:

- **U.S. Environmental Protection Agency: Vehicle Fuel Economy Information**
  FuelEconomy.gov is a U.S. government site that provides official vehicle fuel economy data to help consumers make informed choices. Data is managed by the Department of Energy and EPA.

  https://fueleconomy.gov/feg/ws/index.shtml

Bronze Layer

Overview: Build a raw ingestion pipeline for FuelEconomy.gov's vehicle dataset containing fuel-economy attributes for all model years.

1. **Download CSV file**
   - **Action:** Retrieve the vehicle CSV from FuelEconomy.gov and store it locally prior to ingestion.
     Ensures a consistent external data snapshot for reproducible loads.
2. **Create DBFS RAW folder & move file**
   - **Action:** Create /mnt/epa/raw and move the CSV into the raw zone.
     Establishes a structured file layout for pipeline automation.
3. **Read raw CSV and inspect schema**
   - **Action:** Load the CSV using inferSchema=True for an initial Spark interpretation.
     Provides a first look at column types and data integrity prior to cleaning.
4. **Add Data_update_timestamp**
   - **Action:** Append a timestamp column using current_timestamp().
     Records ingestion time for lineage, auditing, and incremental loading.
5. **Normalize column names**
   - **Action:** Replace invalid characters with underscores.
     Ensures compatibility with Spark SQL, Delta Lake, and downstream ETL.
6. **Write Bronze Delta table**
   - **Action:** Save DataFrame to dbfs:/mnt/epa/bronze using Delta with schema merge enabled.
     Creates an immutable, ACID-compliant raw dataset for all later layers.

7. **Register Bronze table in the metastore**
    - **Action:** Create epa_vehicle.bronze referencing the Delta path.
      Enables SQL access and consistent referencing in future transformations.
8. **Preview Bronze table**
    - **Action:** Display the table to verify schema and record counts.
      Confirms successful ingestion and proper raw zone setup.

Silver Layer

Overview: Clean, standardize, and validate the Bronze dataset while preparing it for analytical usage.

1. **Load Bronze data**
    - **Action:** Read epa_vehicle.bronze into a DataFrame.
      Establishes the raw starting point for Silver-level processing.
2. **Trim whitespace from string columns**
    - **Action:** Apply trim() to every string field.
      Eliminates formatting inconsistencies that affect joins, grouping, and type conversions.
3. **Convert createdOn and modifiedOn to timestamps**
    - **Action:** Use to_timestamp() to parse both fields.
      Enables proper chronology, filtering, and time-based analytics.
4. **Check for NULL timestamps**
    - **Action:** Count records with null createdOn/modifiedOn.
      Identifies incomplete or malformed records requiring attention.
5. **Validate uniqueness of id**
    - **Action:** Test for null id values and identify duplicate ids.
      Confirms the integrity of the dataset's primary identifier.
6. **Count nulls in every column**
    - **Action:** Aggregate null counts per column.
      Establishes an overall data-quality profile for the dataset.
7. **Perform quick numeric profiling**
    - **Action:** Run describe() on key numeric fields such as CO2, displ, and barrels.
      Highlights unusual values, ranges, or distributions requiring validation.
8. **Write Silver Delta table**
    - **Action:** Save DataFrame to dbfs:/mnt/epa/silver with mergeSchema.
      Produces a cleaned, standardized dataset ready for modeling and aggregation.
9. **Register Silver table in the metastore**

- **Action:** Create epa_vehicle.silver referencing the Silver Delta path.
  Makes the standardized dataset available for SQL queries and BI tools.

10. **Preview Silver table**

    - **Action:** Display the table.
      Confirms cleanup, structure, and readiness for Gold processing.

Gold Layer

Overview: Produce a fully validated, analytics-ready dataset with corrected numeric values and optimized structure.

1. **Clean and validate numeric metrics**

    - **Action:** Replace placeholder scores, convert meaningless zeros to nulls, ensure non-negative values on core numeric fields.
      Creates reliable numeric attributes for reporting, dashboards, and analysis.

2. **Reorder columns with id first**

    - **Action:** Move id to the first position in the schema.
      Improves readability and centralizes the primary key.

3. **Identify columns fully null**

    - **Action:** Scan for fields containing only null values.
      Confirms that no empty fields require removal or rework.

4. **Write Gold Delta table with schema enforcement**

    - **Action:** Save Gold dataset to dbfs:/mnt/epa/gold with schema overwrite enabled, then register epa_vehicle.gold.
      Produces the consumer-ready dataset for analytics, BI dashboards, and final reporting.

8. Automation Strategy

Manual API ingestion (critical)

- Data is downloaded manually, validated for quality, and uploaded to DBFS or cloud storage.
- This prevents blocked pipelines or lost time caused by API downtime, corrupted files, rate limits, or unexpected changes.
- Manual checks ensure only clean, complete data enters the system.

Automated Databricks pipeline

- Once uploaded, data flows automatically through Bronze → Silver → Gold layers using Databricks Jobs or Delta Live Tables (DLT).
- Transformations include cleaning, enrichment, aggregation, and incremental updates.
- Scheduling and triggers ensure pipelines run reliably on a fixed cadence or when new files arrive.
- Monitoring, logging, and alerts catch errors in the automated steps.

Reasoning:

- Manual ingestion safeguards against external API issues.
- Automation handles everything we can control—transformations, Medallion layering, and reporting—efficiently and reliably.
- Hybrid approach ensures robust, scalable, and monitorable workflows while minimizing downtime.

Why full automation (automatically downloading source data from public APIs) is not recommended for this project:

- API Reliability Issues: Public APIs can be unstable, experience downtime, or change without notice. Automated ingestion risks pipeline failures if the API is unavailable or returns corrupted data.
- Data Quality Control: Manual download allows you to validate and check the data before processing. Automated ingestion could load incomplete or incorrect data, blocking downstream analytics.
- Debugging & Monitoring: If something breaks in a fully automated system, it can be harder to identify and fix the issue quickly. Manual steps help isolate problems before they affect the entire pipeline.
- API Limitations: Many public APIs have rate limits, access restrictions, or may not support large-scale automated downloads reliably.

- Avoiding Data Blockages: Manual ingestion ensures that only validated, correct data enters the pipeline, reducing the risk of data blockages or delays due to API problems.

Summary:
Manual download and upload of source data provide better control, reliability, and flexibility. Automation is applied only after data is validated and available in storage, ensuring robust and efficient processing in Databricks.

9. Challenges & Solutions

API Ingestion Challenges

- Pulling data directly from the API into Spark was unreliable because of timeouts, network errors, and service instability.
- A separate script pulling API data into cloud storage proved more stable and easier to manage.
- Saving data in a simple file format made Databricks ingestion smoother.
- Using Autoloader allowed new files to be picked up automatically without manual triggers.

Large Initial Load Challenges

- The initial ZIP file was too large and difficult to import through the API, so downloading it manually was more dependable.
- PowerQuery helped split the huge dataset into smaller CSVs so nothing was lost and the files were manageable in Databricks.
- Breaking the file into three parts made Bronze ingestion more reliable and prevented memory issues.

Solutions – ZIP Handling Pipeline

- **Streaming the download** in small chunks kept memory usage low and avoided crashes.
- **Extracting with Python's zipfile** provided a clean way to prepare the raw data for Spark.
- **Chunk-based streaming** (iter_content(8192)) ensured the file was processed safely without loading it all into memory.

Handling null values in datasets challenge where there is no direct contact with the data owner. Dropping columns is not an option since they might still be needed for downstream analysis or transformations.

Solution:
Use type-appropriate safe defaults to replace all null values while retaining all columns:

- **Strings:** Replace nulls with "UNKNOWN" to indicate missing textual data without losing the column.
- **Integers:** Replace nulls with -1 to signal missing numeric values safely.
- **Floats/Decimals:** Replace nulls with -1.0 to maintain numeric consistency and allow calculations.
- **Boolean:** Replace nulls with False to provide a default logical value.
- **Other types:** Replace nulls with None to preserve column structure without introducing invalid values.

Benefit:
This approach keeps the dataset complete, avoids dropping potentially important columns, and ensures downstream transformations, analytics, and modeling can proceed reliably without encountering null-related errors.

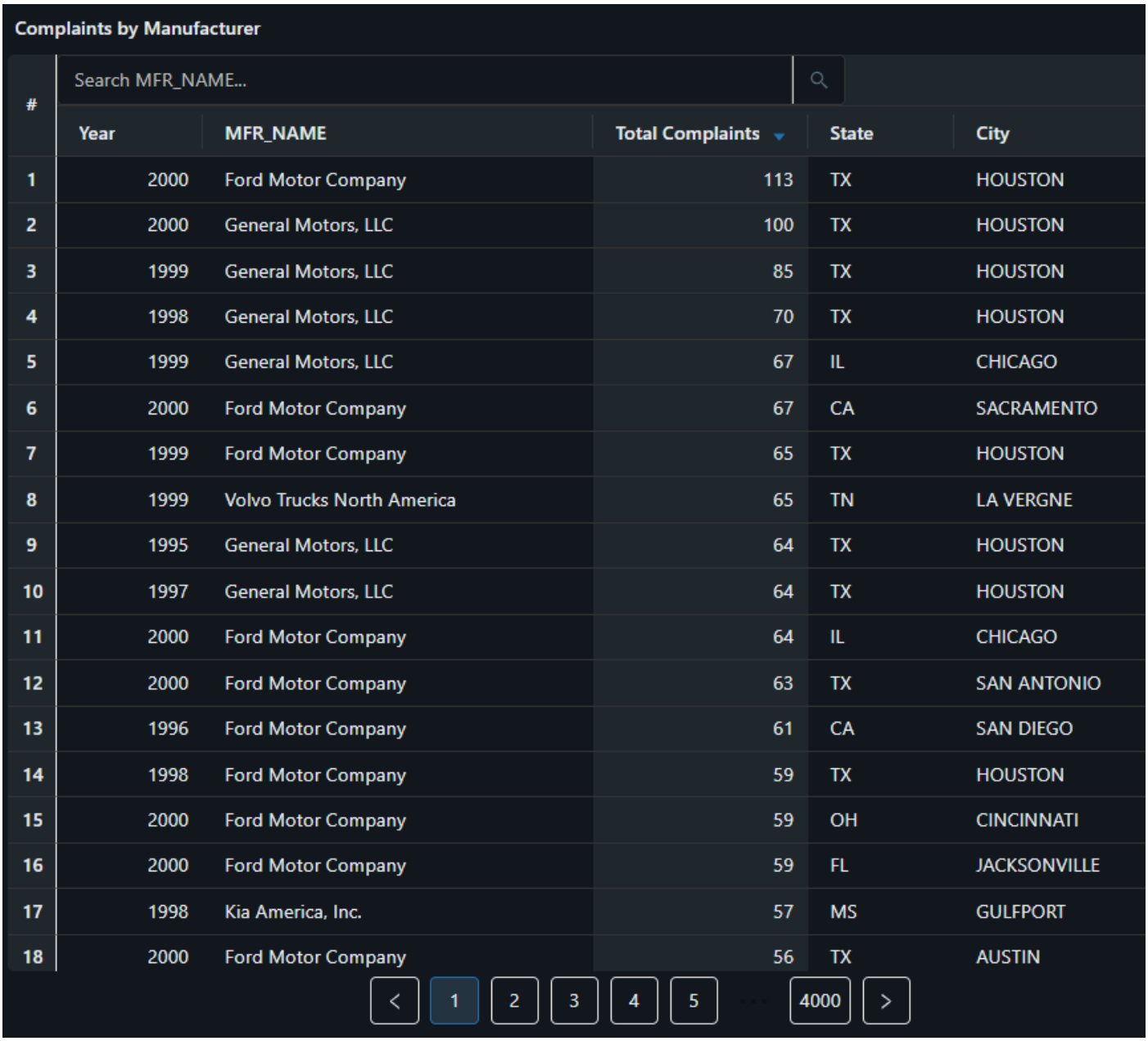
10.Dashboard Insights via Databricks Dashboards

Leveraging the final Gold-layer datasets from my data sources, I took the opportunity to build three dashboards that deliver actionable insights and KPIs for automotive and transportation decision-making. These dashboards allow leadership, managers, and sustainability teams to monitor key metrics and make informed business decisions.

NHTSA Complaints Dashboard:

Complaints by Manufacturer

Business Decision: Focus quality improvement initiatives on manufacturers with high complaint counts.



Complaints by Manufacturer

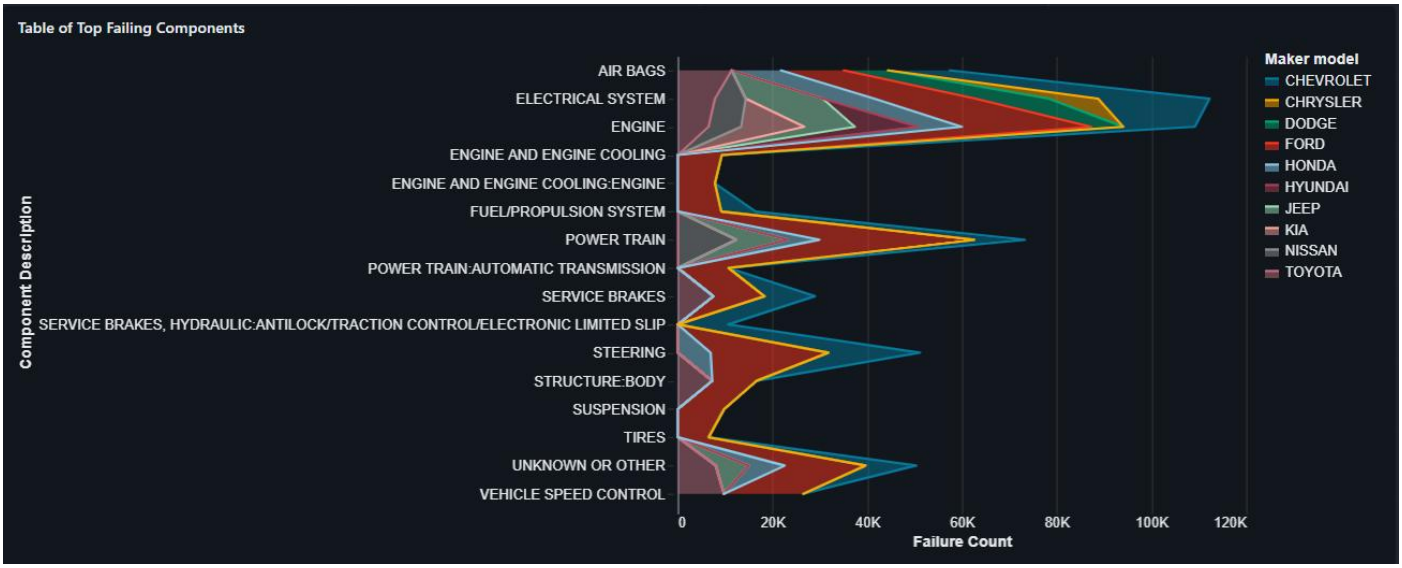
Search MFR_NAME...

#	Year	MFR_NAME	Total Complaints	State	City
1	2000	Ford Motor Company	113	TX	HOUSTON
2	2000	General Motors, LLC	100	TX	HOUSTON
3	1999	General Motors, LLC	85	TX	HOUSTON
4	1998	General Motors, LLC	70	TX	HOUSTON
5	1999	General Motors, LLC	67	IL	CHICAGO
6	2000	Ford Motor Company	67	CA	SACRAMENTO
7	1999	Ford Motor Company	65	TX	HOUSTON
8	1999	Volvo Trucks North America	65	TN	LA VERGNE
9	1995	General Motors, LLC	64	TX	HOUSTON
10	1997	General Motors, LLC	64	TX	HOUSTON
11	2000	Ford Motor Company	64	IL	CHICAGO
12	2000	Ford Motor Company	63	TX	SAN ANTONIO
13	1996	Ford Motor Company	61	CA	SAN DIEGO
14	1998	Ford Motor Company	59	TX	HOUSTON
15	2000	Ford Motor Company	59	OH	CINCINNATI
16	2000	Ford Motor Company	59	FL	JACKSONVILLE
17	1998	Kia America, Inc.	57	MS	GULFPORT
18	2000	Ford Motor Company	56	TX	AUSTIN

< 1 2 3 4 5 ... 4000 >

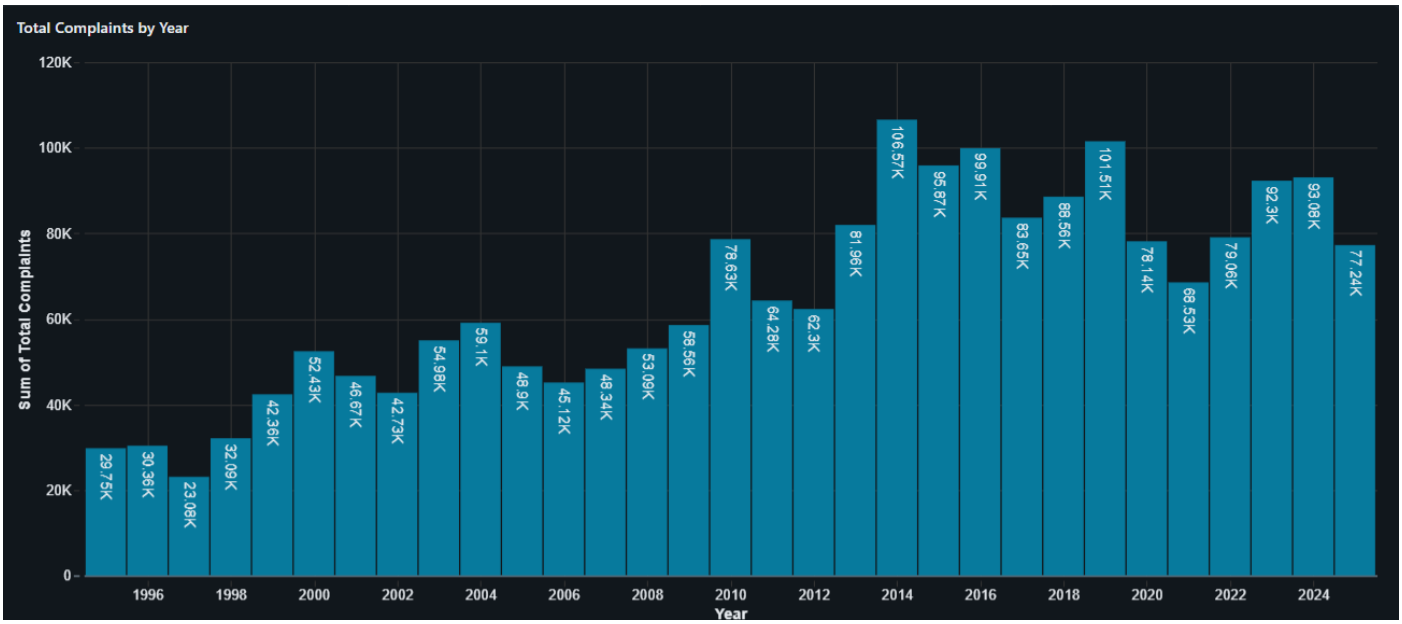
Top Failing Components

Business Decision: Prioritize design or recall interventions for the most frequently failing parts.



Total Complaints by Year

Business Decision: Identify trends over time to plan proactive maintenance, recalls, or safety campaigns.



SQL query used:

Total complaints by year and manufacturer

SELECT YEAR(FAILDATE) AS year, MFR_NAME, State, City, COUNT(*) AS total_complaints FROM hive_metastore.nhtsa_complaints.gold WHERE YEAR(FAILDATE) BETWEEN 1995 AND 2025 GROUP BY YEAR(FAILDATE), MFR_NAME, State, City ORDER BY year, total_complaints DESC

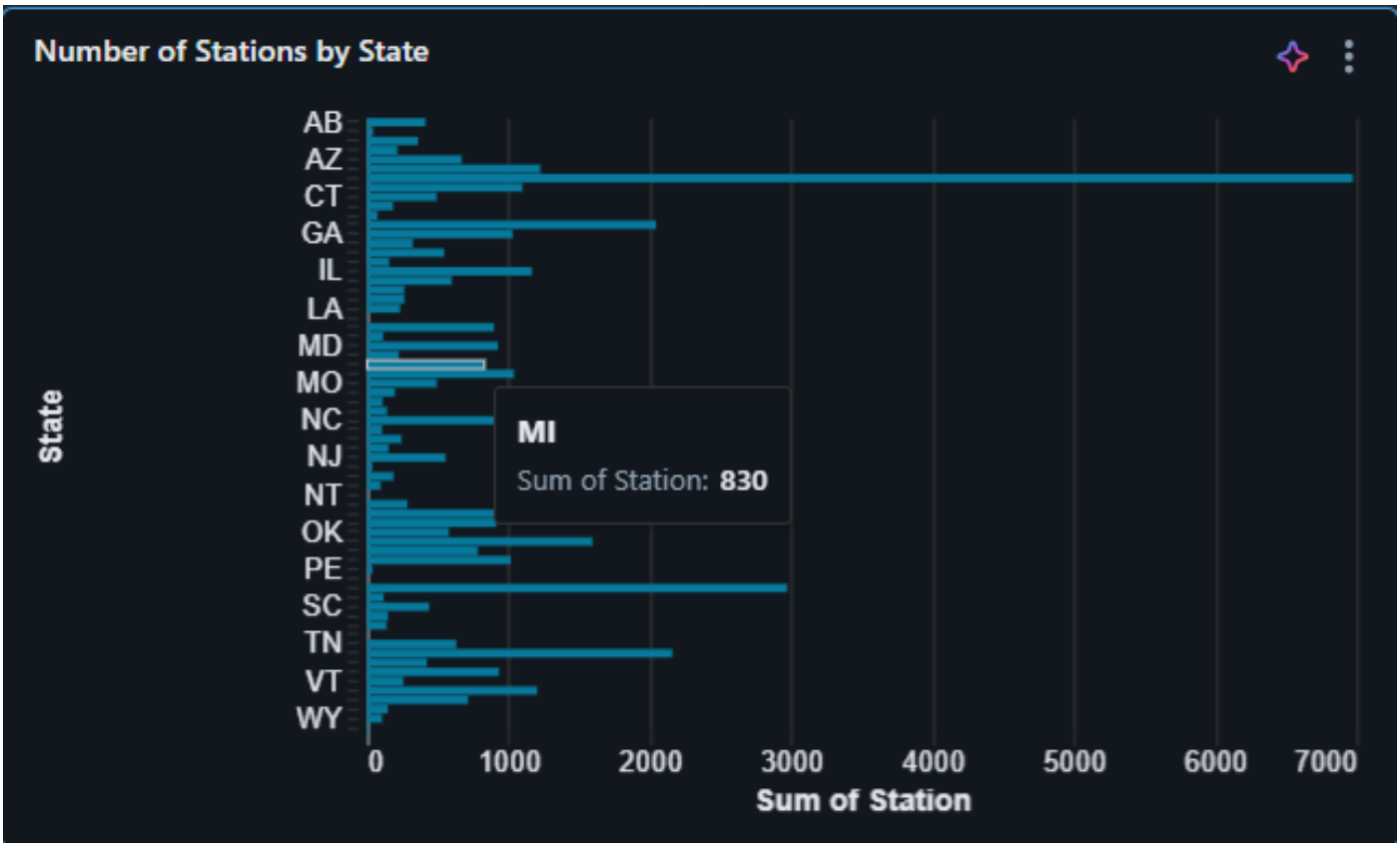
Top failing components

SELECT COMPDESC, COUNT(*) AS failure_count, MAKETXT, any_value(MODELTXT) AS MODELTXT FROM hive_metastore.nhtsa_complaints.gold GROUP BY COMPDESC, MFR_NAME, MAKETXT ORDER BY failure_count DESC LIMIT 50;

AFDC - Alternative Fuel Stations

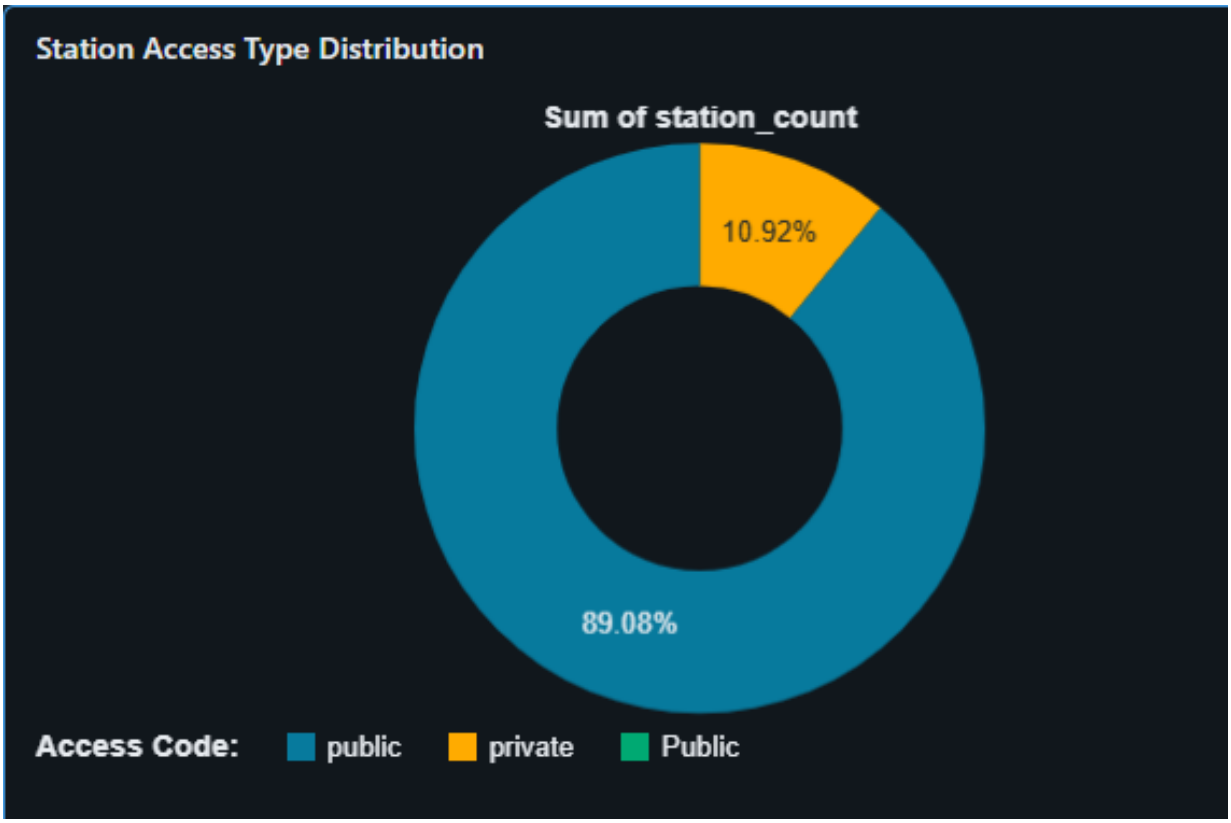
Number of stations by state

Business Decision: Expand stations in states or fuel types with low coverage.



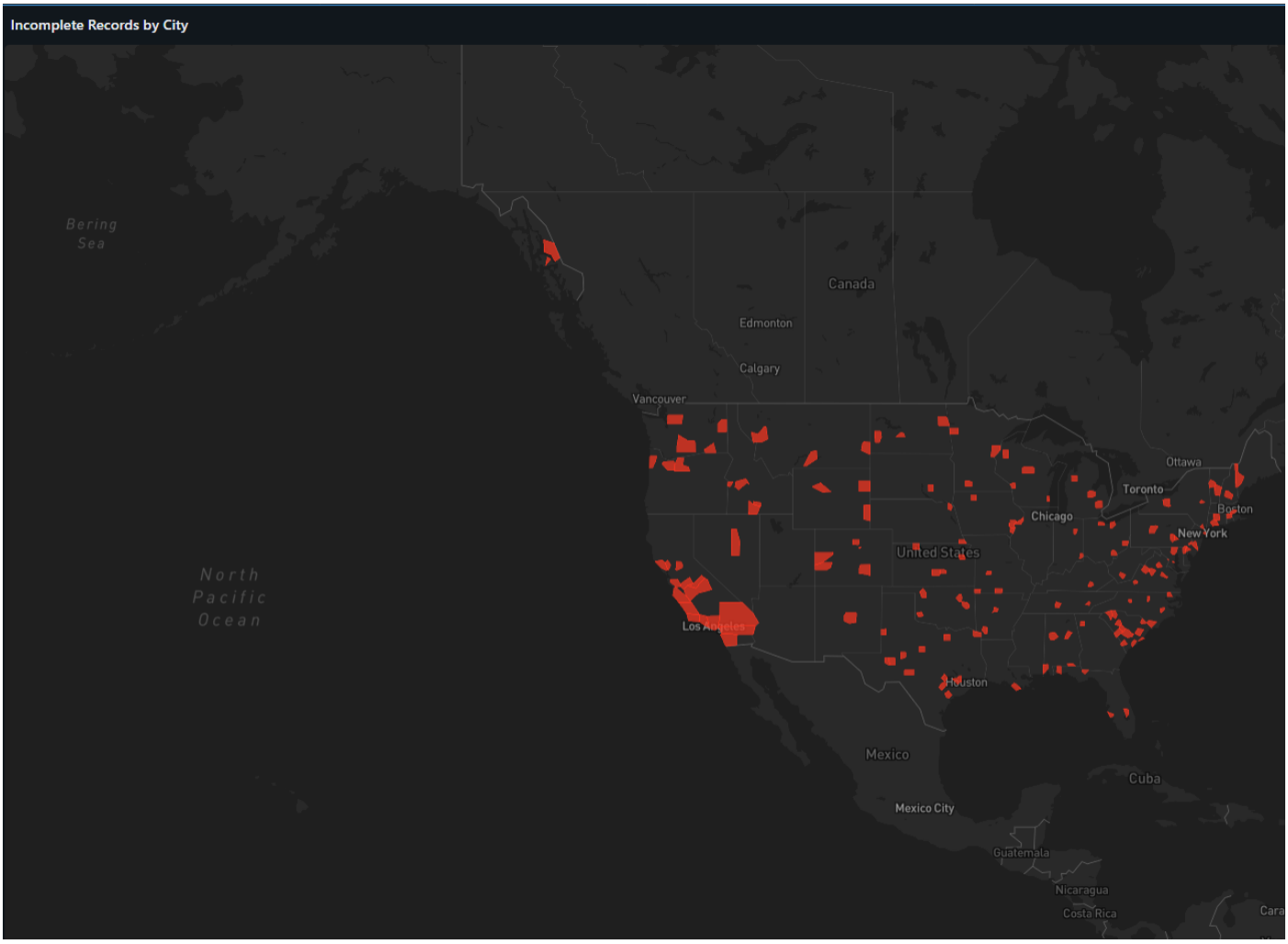
Station access type distribution

Business Decision: Increase public access stations where restricted access dominates.



Incomplete records by city

Business Decision: Improve data quality to support accurate planning and operations.



SQL query used:

Number of stations by state and fuel type

```
SELECT
    State,
    CASE Fuel_Type_Code
        WHEN 'BD' THEN 'Biodiesel_B20'
        WHEN 'CNG' THEN 'Compressed_Natural_Gas'
        WHEN 'ELEC' THEN 'Electric'
        WHEN 'E85' THEN 'Ethanol'
        WHEN 'HY' THEN 'Hydrogen'
        WHEN 'LNG' THEN 'Liquefied_Natural_Gas'
        WHEN 'LPG' THEN 'Propane'
```

```
        WHEN 'RD' THEN 'Renewable_Diesel'
        ELSE Fuel_Type_Code
    END AS Fuel_Type_Description,
    COUNT(*) AS station_count
FROM adfc_alternative_fuel_stations.gold
GROUP BY State, Fuel_Type_Code
ORDER BY State, station_count DESC;
```

Stations by access type (public vs restricted)

```
SELECT Access_Code AS access_code, COUNT(*) AS station_count
FROM adfc_alternative_fuel_stations.gold
GROUP BY Access_Code;
```

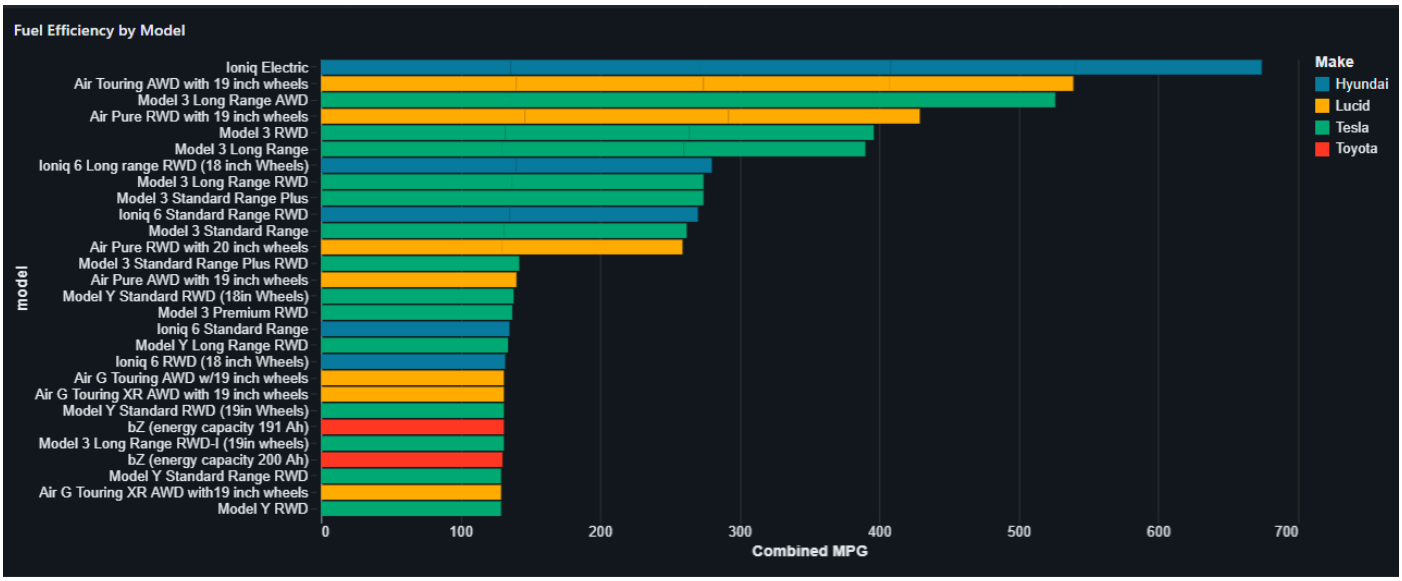
Stations with incomplete information

```
SELECT State, City, COUNT(*) AS incomplete_records
FROM adfc_alternative_fuel_stations.gold
WHERE Station_Phone = 'Unknown' OR Access_Code = 'Unknown'
GROUP BY State, City
ORDER BY incomplete_records DESC;
```

EPA - Vehicle Fuel Economy Information

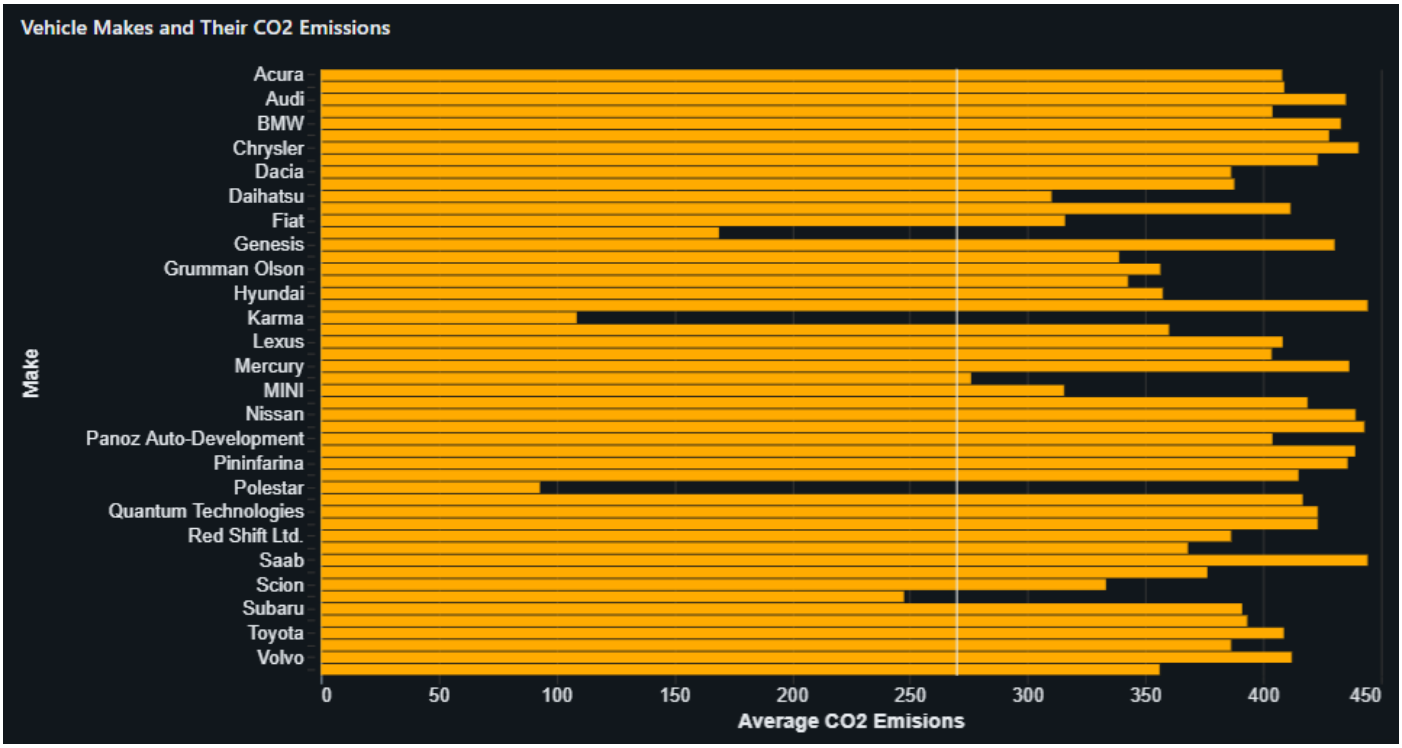
Fuel Efficiency by Model

Business Decision: Promote or prioritize high-MPG models to reduce fuel costs and support sustainability goals.



Average Annual Fuel Cost by Vehicle Class

Business Decision: Guide fleet or consumer purchases toward vehicle classes with lower operating costs.



Vehicle Makes and Their CO2 Emissions

Business Decision: Target low-emission manufacturers for eco-friendly fleet expansion or marketing campaigns.